

Impacto de la disponibilidad de ChatGPT en el desarrollo de software orientado a Ciencia de Datos (2020–2023)

Sustentación de tesis para optar al título de Licenciado en Economía

Jesus Alberto Nicho Rosado
Karl Willem Janampa Aparicio

Pontificia Universidad Católica del Perú
Facultad de Ciencias Sociales
Asesor: Alexander Wilder Quispe Rojas

Lima, 2026

1. Motivación y problemática
2. Pregunta e hipótesis
3. Marco teórico
4. Evidencia empírica previa
5. Línea de tiempo de ChatGPT
6. Datos y diseño del panel
7. Metodología (DiD, CS, SDiD)
8. Resultados principales
9. Pruebas de robustez
10. Limitaciones y conclusiones

1. Motivación

- ChatGPT (OpenAI, 30 noviembre 2022) marcó un punto de inflexión en la asistencia con IA generativa para tareas de programación.
- Su escala fue cualitativamente distinta a tecnologías previas (Copilot, Codex): **servicio gratuito, masivo y conversacional**.
- Su **disponibilidad geográfica heterogénea** —bloqueado en China, Rusia, Irán, Cuba, Corea del Norte, entre otros— generó un escenario cuasi-experimental natural.
- El sector software se proyectaba a USD 600 000 millones para 2025 (Statista, 2023). Toda mejora marginal en productividad tiene consecuencias agregadas relevantes.

La pregunta práctica

Si ChatGPT efectivamente aumenta la productividad o la entrada de desarrolladores, sus implicaciones cruzan tres dimensiones:

Política pública

Subsidios al acceso, regulación de servicios IA, restricciones internacionales.

Empresas

Inversión en suscripciones premium, programas de adopción, evaluación de productividad.

Educación

Currículos de ciencia de datos, formación de juniors, certificaciones laborales.

Sin evidencia causal robusta, las decisiones se toman a ciegas.

2. Pergunta e hipótesis

Pregunta central

¿La **disponibilidad de ChatGPT** causó un incremento del número de desarrolladores activos en GitHub en los países con acceso, comparado con los países donde el servicio fue restringido, entre 2020 y 2023?

Unidad de análisis: país $\times$ lenguaje de programación $\times$ trimestre.

Tratamiento: acceso público a ChatGPT a partir de noviembre 2022.

Variable de resultado: *unique pushers* por cada 100 000 habitantes.

Período: Q1-2020 a Q4-2023 (16 trimestres).

- **H1.** La accesibilidad a ChatGPT se asocia positivamente con el desarrollo de software en los países tratados.
- **H2.** El efecto es **heterogéneo por lenguaje**: más fuerte en Python y JavaScript dada su vasta documentación técnica disponible en internet, recurso fundamental para el entrenamiento del modelo.

Mecanismos teóricos detrás de las hipótesis

- 1 **Reasignación de tareas** (Acemoglu & Restrepo, 2018, 2019): la IA libera tareas rutinarias, expande la frontera de proyectos rentables y atrae programadores marginales.
- 2 **Reducción de costos de búsqueda y aprendizaje** (Stigler, 1961): ChatGPT colapsa el costo de explorar documentación y foros a niveles cercanos a cero.

3. Marco teórico

Dos fuerzas opuestas sobre el trabajo

La producción es un conjunto de **tareas** que puede realizar el **trabajo** (humano) o el **capital** (algoritmos). Una nueva tecnología empuja en dos direcciones a la vez:

Displacement effect

↓ reduce la demanda de trabajo

Automatiza tareas antes del programador:

- Búsqueda de sintaxis
- Redacción de *boilerplate*
- Depuración rutinaria

Reinstatement effect

↑ aumenta la demanda de trabajo

Al abaratar producir código:

- Proyectos antes inviables se vuelven rentables.
- Programadores marginales pueden entrar.
- La frontera productiva se expande.

¿Cuál fuerza domina? **Nuestra tesis encuentra que domina el *reinstatement*: más desarrolladores, no menos.**

Costos de búsqueda y de aprendizaje

Stigler (1961) modela la búsqueda como un costo que el agente paga para reducir incertidumbre sobre alternativas.

En desarrollo de software, este costo se traduce en:

- Tiempo dedicado a leer documentación oficial.
- Búsquedas en Stack Overflow, foros, tutoriales.
- Experimentación con código que falla.

ChatGPT actúa como un *shock* negativo sobre estos costos: ofrece respuestas contextualizadas e instantáneas.

Evidencia coherente: Del Río-Chanona et al. (2023)

–16 % en actividad de Stack Overflow tras el lanzamiento de ChatGPT $\Rightarrow$ reorganización endógena de los canales de búsqueda.

Marco teórico (III): modelo formal de adopción heterogénea

Variables del modelo

Símbolo	Significado
$z \in [0, 1]$	Tarea elemental de programación
$h_c(z), a_c(z)$	Productividad humana / de ChatGPT en la tarea z
μ_j, A_{it}	Acceso individual / disponibilidad país–trimestre
$\mathcal{Z}_c^{\text{auto}}, \tau_c$	Conjunto de tareas automatizables y su medida (fracción)
$s_c(z), \rho_c$	Ahorro relativo por tarea y su promedio condicional
$\theta_c = \tau_c \rho_c$	Exposición del lenguaje (componente de lenguaje)
η, r_i, η_c^*	Habilidad, utilidad de reserva y umbral de entrada
F_i, f_i	Distribución y densidad de habilidad del país
$\phi_i = 10^5 f_i(\eta_c^*(0))$	Absorción del país (componente de país)
Y_{ict}	<i>Unique pushers</i> por 100 mil habitantes

Marco teórico (III): modelo formal de adopción heterogénea

Del desarrollador individual al umbral de entrada (Pasos 1–2)

Paso 1 — Producción y exposición. En cada tarea el desarrollador usa el método más productivo (humano o ChatGPT). Por eso su **ganancia de productividad** al acceder a la herramienta, Δy_{jc} , proviene *solo* de las tareas que ChatGPT hace mejor; esa ganancia es la **exposición del lenguaje** θ_c :

$$\underbrace{\Delta y_{jc}}_{\text{ganancia de productividad}} = \underbrace{\tau_c}_{\text{fracción de tareas automatizables (amplitud)}} \cdot \underbrace{\rho_c}_{\text{ahorro relativo promedio (profundidad)}} \equiv \theta_c.$$

Combina *cuántas* tareas mejora ChatGPT (τ_c) y *cuánto* ahorra en ellas (ρ_c); es mayor en lenguajes muy representados en los datos de entrenamiento (Python, JavaScript, TypeScript).

Paso 2 — Umbral de entrada. Cada persona tiene una **habilidad** η (propia de cada individuo, varía entre personas) y entra a programar si supera el **umbral** η_c^* (común a todos: la barra mínima de entrada):

$$\text{entra si } \underbrace{\eta}_{\text{habilidad del individuo}} \geq \underbrace{\eta_c^*}_{\text{barra mínima de entrada}}.$$

La ganancia de productividad **baja esa barra** exactamente en θ_c : $\eta_c^*(1) = \eta_c^*(0) - \theta_c$.
Entran quienes tienen habilidad entre la barra nueva y la antigua: los *desarrolladores marginales*.

Marco teórico (III): modelo formal de adopción heterogénea

De la agregación poblacional al panel estimable (Pasos 3–4)

Paso 3 — Agregación y absorción. Cuando la barra baja en θ_c , entran las personas que estaban justo debajo. ¿Cuántas? Depende de la **densidad** de gente cerca del umbral (f_i). Por cada 100 mil habitantes:

$$\underbrace{\Delta Y_{ic}}_{\text{nuevos devs. por 100 mil hab.}} \approx \underbrace{\phi_i}_{\text{absorción del país}} \cdot \underbrace{\theta_c}_{\text{exposición del lenguaje}}, \quad \phi_i = 10^5 f_i(\eta_c^*(0)).$$

Intuición: nuevos $\approx$ (densidad cerca del umbral) $\times$ (cuánto bajó la barra). Por eso ϕ_i es la densidad f_i evaluada en el umbral.

Paso 4 — Especificación de panel. Para estimarlo con datos, la variable observada Y_{ict} (*unique pushers*) se descompone en efectos fijos que **controlan** diferencias permanentes de país (α_i), trimestre (β_t) y lenguaje (δ_c), más el efecto que aparece solo cuando ChatGPT está disponible ($A_{it} = 1$):

$$Y_{ict} = \alpha_i + \beta_t + \delta_c + \underbrace{\theta_c \phi_i}_{\text{efecto del tratamiento}} A_{it} + \eta_{ict}.$$

$A_{it} = 1$ si el país i tiene acceso en el trimestre t ; η_{ict} : factores no observados. El efecto es **multiplicativo**: máximo donde coinciden alta exposición y alta absorción.

Marco teórico (III): modelo formal de adopción heterogénea

Supuestos del modelo

- 1 **Mejor método por tarea:** el desarrollador usa el método más productivo en cada tarea.
- 2 **Covarianza nula:** $\text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) = 0$ — la productividad humana y el ahorro relativo no están correlacionados dentro de las tareas automatizables.
- 3 **Normalización de escala:** $\bar{h}_c = 1$ (elección de unidades de producción).
- 4 **Taylor de primer orden:** válido si θ_c es pequeño frente a la escala en que varía la densidad f_i .
- 5 **Separabilidad país–lenguaje:** $\phi_{ic} \approx \phi_i$, lo que permite factorizar el coeficiente como $\theta_c \phi_i$.
- 6 **Exogeneidad condicional:** $\mathbb{E}[\eta_{ict} \mid \alpha_i, \beta_t, \delta_c, A_{it}] = 0$ (identificación causal).

Marco teórico (III): modelo formal de adopción heterogénea

Conclusiones del modelo

El efecto de tratamiento es $\Delta Y_{ic} = \theta_c \phi_i$. **Estática comparativa** — cómo cambia ese efecto al mover cada componente:

- Más **exposición** del lenguaje $\Rightarrow$ más efecto: $\frac{\partial \Delta Y_{ic}}{\partial \theta_c} = \phi_i > 0$.
- Más **absorción** del país $\Rightarrow$ más efecto: $\frac{\partial \Delta Y_{ic}}{\partial \phi_i} = \theta_c > 0$.
- **Se refuerzan mutuamente:** $\frac{\partial^2 \Delta Y_{ic}}{\partial \theta_c \partial \phi_i} = 1 > 0$. El efecto de más exposición es mayor cuando la absorción es alta; no se suman, se *multiplican* (complementariedad).

Predicción contrastable (sustenta la H2)

El ATT debe ser mayor en lenguajes muy expuestos (Python, JavaScript, TypeScript) y en países con más población cerca del umbral de entrada.

4. Evidencia empírica

Evidencia empírica: impacto económico de la IA

Autor	Efecto	Resultado breve
Brynjolfsson et al. (2023)	↑ Positivo	↑ 14 % la productividad en atención al cliente, especialmente en empleados con menor experiencia.
Gallea (2023)	↑ Positivo	↓ preguntas comunes en Stack Overflow y orienta a los usuarios hacia consultas más complejas.
Wu et al. (2024)	↑ Positivo	↑ valor de mercado de las empresas chinas que decidieron integrar la IA.
Liu et al. (2023)	≈ Neutro	↓ la demanda de trabajos automatizables, pero favoreció a freelancers que lo integraron en su servicio.
Del Río-Chanona et al. (2023)	↓ Negativo	↓ uso de foros como Stack Overflow, afectando la generación de datos públicos para entrenar IA.
Demirci et al. (2023)	↓ Negativo	↓ demanda de tareas automatizables y de generación de imágenes.
Hui et al. (2023)	↓ Negativo	↓ el empleo y los ingresos de <i>freelancers</i> en ocupaciones vulnerables como redacción y edición.

Evidencia empírica: capacidades por lenguaje de programación

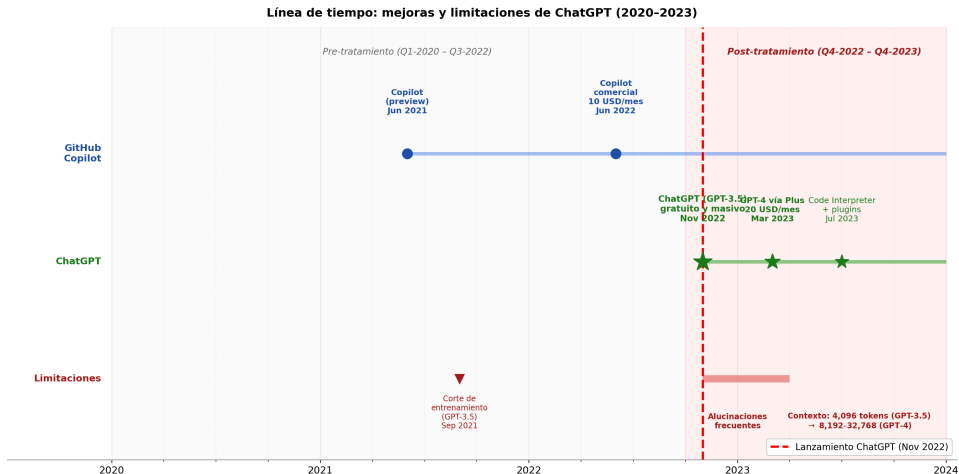
Autor	Efecto	Resultado breve
Buscemi (2023)	↑ Positivo	En ciencia de datos, Julia mostró el mejor desempeño (81,5 %), seguido de R y Python, destacando la eficacia del modelo en lenguajes populares y especializados en análisis estadístico.
Chai et al. (2024)	↑ Positivo	Usando el <i>benchmark</i> MCEVAL, se evidenció que el rendimiento de ChatGPT-3.5 Turbo varía ampliamente entre lenguajes: destaca en Python, Java y Ruby, pero muestra limitaciones en lenguajes menos comunes como Lisp, Scheme y VimL.
Li y Krishnamachari (2024)	↑ Positivo	MultiPL-LeetCode mostró que el rendimiento de ChatGPT-3.5 Turbo varía según el lenguaje: es alto en Python, Java y C++, pero deficiente en lenguajes funcionales como Elixir, Erlang y Racket.

Brecha en la literatura

Ningún estudio analiza específicamente el impacto causal de ChatGPT sobre el **desarrollo de software** desagregado por lenguaje a nivel de país.

5. Línea de tiempo

Línea de tiempo: ChatGPT y limitaciones (2020–2023)



Tres carriles: GitHub Copilot (azul), ChatGPT (verde), Limitaciones (rojo, duraciones).

La ventana postratamiento (Q4-2022 a Q4-2023) no fue homogénea:

Q4-2022 – Q1 2023

- GPT-3.5 gratuito
- Alucinaciones frecuentes
- Contexto 4 096 tokens
- Corte de entrenamiento: Sep 2021

Q2-Q4 2023

- GPT-4 vía Plus (20 USD/mes)
- Reducción de alucinaciones
- Contexto 8 192–32 768 tokens
- Code Interpreter, *plugins*

Esto abre la posibilidad de un efecto no monotónico en el corto plazo.

6. Datos

Fuente: GitHub Innovation Graph

Base pública mantenida por GitHub con métricas agregadas de actividad de desarrollo por país, lenguaje y trimestre.

Variable de resultado: *unique pushers* por cada 100 000 habitantes.

Unique pushers = cantidad de desarrolladores únicos que al menos hicieron un `git push` en GitHub durante un trimestre, normalizado por población.

- **Mide:** entrada/persistencia de programadores activos.
- **No mide:** intensidad por persona, calidad del código, valor agregado.
- **Limitación reconocida:** países como China usan plataformas alternativas (Gitee) $\Rightarrow$ posible subestimación en el control.

Dimensiones del panel:

- **159 países** (N): 130 tratados + 29 control.
- **10 lenguajes**: C, C#, C++, Go, Java, JavaScript, PHP, Python, Ruby, TypeScript.
- **16 trimestres**: Q1-2020 a Q4-2023.
- **Total**: 25 440 observaciones.

Período de pretratamiento: 12 trimestres (Q1-2020 a Q3-2022).

Período de tratamiento: 4 trimestres (Q4-2022 a Q4-2023).

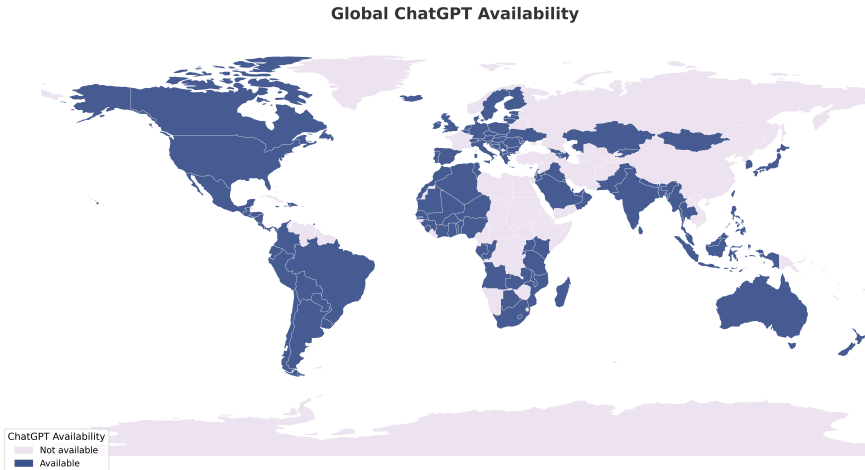
Exclusiones: EU (agregado), Kosovo (sin código ISO procesable), Hong Kong (atípico).

Países sin acceso a ChatGPT (control)

Censores severos: China, Cuba, Irán, Bielorrusia, Rusia, Siria.

Otros: Afganistán, Bahréin, Burundi, Camboya, Camerún, Congo (RD), Egipto, Eswatini, Etiopía, Guadalupe, Macao, Puerto Rico, Reunión, Somalia, Sudán, Tayikistán, Turkmenistán, Turquía, Uzbekistán, Venezuela, Vietnam, Yemen, Zimbabue, Libia, Laos.

Mapa global de disponibilidad de ChatGPT



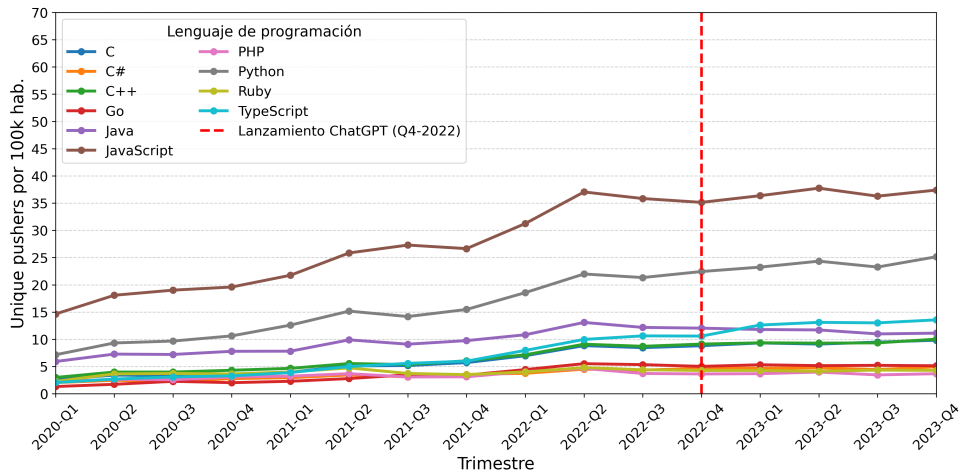
Azul: países con acceso (tratados, $N = 130$). **Gris claro:** países sin acceso (control, $N = 29$).

Media de *unique pushers* por 100k habitantes

Lenguaje	Pretratamiento		Postratamiento	
	Control	Tratado	Control	Tratado
JavaScript	18,84	51,00	34,29	58,98
Python	13,30	22,71	22,21	33,46
TypeScript	3,46	17,32	11,79	18,83
Java	6,59	13,58	10,81	16,14
C	4,88	7,16	8,73	10,91
C++	5,28	7,77	8,84	10,82
C#	3,01	7,39	4,41	9,64
Go	1,74	3,63	4,85	4,05
Ruby	3,13	6,07	4,00	6,28
PHP	2,27	7,54	3,46	8,24

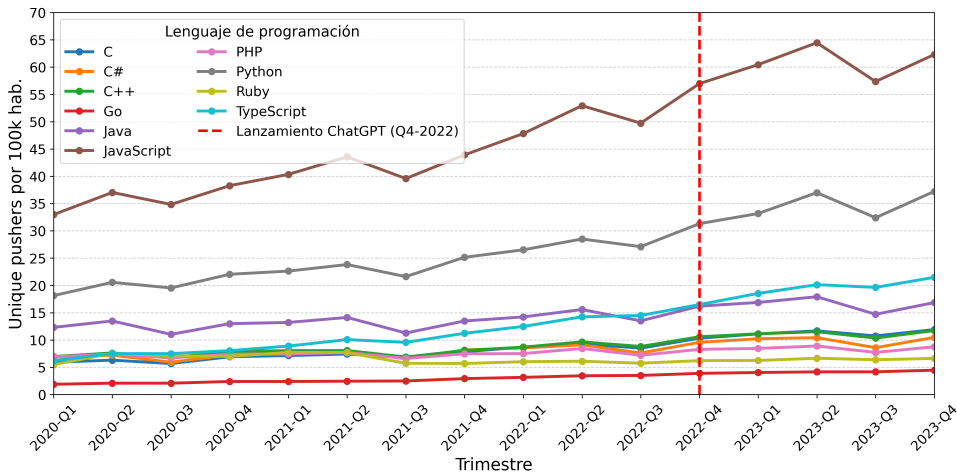
Lectura: ambos grupos crecen en el período. El efecto causal requiere comparar las **tasas** de cambio, no los niveles.

Tendencias temporales (países sin ChatGPT)



Línea vertical roja: lanzamiento de ChatGPT (Q4-2022). **En el grupo de control las series no muestran aceleración tras el tratamiento.**

Tendencias temporales (países con ChatGPT)



Línea vertical roja: lanzamiento de ChatGPT (Q4-2022). **Atención a la aceleración de JavaScript, Python y TypeScript post-tratamiento.**

7. Metodología

Marco común: resultados potenciales

Para cada unidad i en el período t :

$$Y_{it} = Y_{it}^{(0)} + \tau_{it} W_{it}$$

donde $W_{it} = \mathbf{1}[i \in \mathcal{T}, t \geq T_0 + 1]$.

Parámetro de interés: efecto promedio del tratamiento sobre los tratados (ATT):

$$\tau = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \tau_{it}$$

Tres estrategias para estimar τ

- 1 **DiD** — Diferencias en Diferencias (Card & Krueger, 1994).
- 2 **CS** — Control Sintético (Abadie, Diamond & Hainmueller, 2010).
- 3 **SDiD** — Diferencias en Diferencias Sintética (Arkhangelsky et al., 2021). **Método principal.**

Especificación de efectos fijos de dos vías:

$$Y_{it} = \mu + \alpha_i + \beta_t + \tau^{DiD} W_{it} + \varepsilon_{it}$$

Estimador explícito:

$$\hat{\tau}^{DiD} = (\bar{Y}_{post, \mathcal{T}} - \bar{Y}_{pre, \mathcal{T}}) - (\bar{Y}_{post, \mathcal{C}} - \bar{Y}_{pre, \mathcal{C}})$$

Supuesto crítico: tendencias paralelas pre-tratamiento.

Debilidad para este estudio: el control es heterogéneo (países autoritarios) y las tendencias pretratamiento podrían no ser paralelas.

Método 2: Control Sintético (CS)

Estimador:

$$\hat{\tau}^{SC} = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \left(Y_{it} - \sum_{j \in \mathcal{C}} \hat{\omega}_j^{sc} Y_{jt} \right)$$

Pesos $\hat{\omega}_j^{sc}$:

$$\hat{\omega}^{sc} = \arg \min_{\omega} \sum_{t=1}^{T_0} \left(\sum_{j \in \mathcal{C}} \omega_j Y_{jt} - \bar{Y}_t^T \right)^2 \quad \text{s.a.} \quad \omega_j \geq 0, \sum_j \omega_j = 1$$

Idea: construir un “tratado sintético” como promedio ponderado de los controles, que copie la trayectoria pretratamiento del tratado real.

Ventaja: maneja heterogeneidad en el grupo de control.

Limitación: mayor varianza cuando el tratado es agregado de muchas unidades heterogéneas.

Método 3: SDiD (Arkhangelsky et al., 2021)

Combina DiD y CS asignando pesos a unidades y a períodos:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \min \sum_{i,t} \hat{\omega}_i^{sdid} \hat{\lambda}_t^{sdid} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it} \tau)^2$$

Pesos unitarios $\hat{\omega}^{sdid}$: resuelven un problema CS con regularización.

Pesos temporales $\hat{\lambda}^{sdid}$: dan más peso a períodos pretratamiento cercanos a T_0 .

Por qué SDiD es el principal

- Asintóticamente normal con muchas unidades tratadas ($N_{tr} = 130$).
- Robusto a violaciones del supuesto de tendencias paralelas.
- Combina la flexibilidad de CS con la simplicidad de DiD.

Inferencia: *bootstrap* por remuestreo de unidades.

- DiD y SDiD: 100 replicaciones.
- CS: 2 000 replicaciones (mayor varianza por iteración).

Pruebas de robustez adicionales:

- 1 **Cuadro 4:** con variables de control (uso de PC y de internet, Banco Mundial).
- 2 **Cuadro 5:** grupo de control restringido (excluye los 6 censores severos de Freedom House).
- 3 **Cuadro 6:** placebos in-space + sensibilidad LOO al pool de donantes (Abadie, 2010, 2021).

8. Resultados

Resultados principales: ATT por método (Cuadro 3)

Lenguaje	DiD	CS	SDiD
JavaScript	13,635*** (2,719)	9,451* (5,307)	8,303*** (3,180)
Python	7,688*** (1,193)	1,604 (3,552)	4,525*** (1,321)
TypeScript	7,181*** (0,832)	3,263 (2,290)	3,078*** (0,873)
Java	2,810*** (0,827)	2,206 (2,089)	1,725** (0,872)
C	2,689*** (0,365)	1,634 (1,137)	1,276*** (0,451)
C++	2,210*** (0,417)	1,212 (0,959)	1,131*** (0,435)
C#	1,499*** (0,381)	1,001 (1,263)	0,443 (0,379)
Go	1,411*** (0,177)	0,686 (0,606)	0,720*** (0,269)
PHP	0,904* (0,507)	1,078 (1,013)	0,916** (0,465)
Ruby	0,262 (0,266)	1,521 (0,946)	0,808*** (0,274)

Errores estándar *bootstrap* entre paréntesis. * $p < 0,10$ ** $p < 0,05$ *** $p < 0,01$.

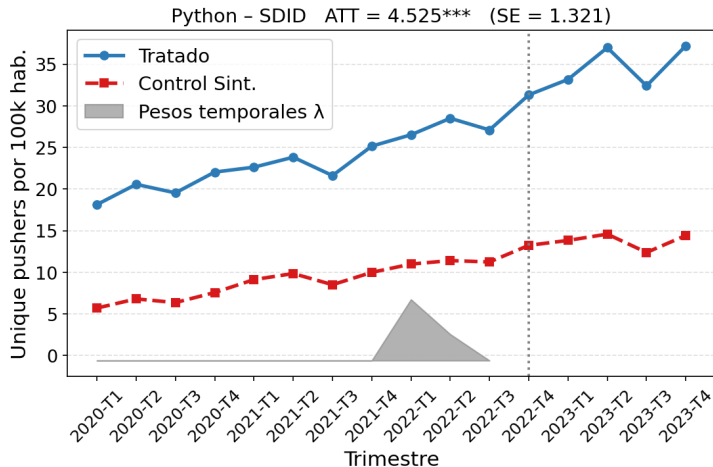
Hallazgos clave del Cuadro 3

- **Nueve de diez lenguajes** muestran ATT positivo y significativo en el SDiD (1 % o 5 %).
- El único lenguaje sin efecto: **C#** (SDiD = 0,443, no significativo).
- Magnitudes más grandes: **JavaScript** (+8,3), **Python** (+4,5), **TypeScript** (+3,1).
- Lenguajes con mayor representación en la web y data science presentan mayores efectos $\Rightarrow$ consistente con la **H2**.

Pero hay que ser cuidadosos

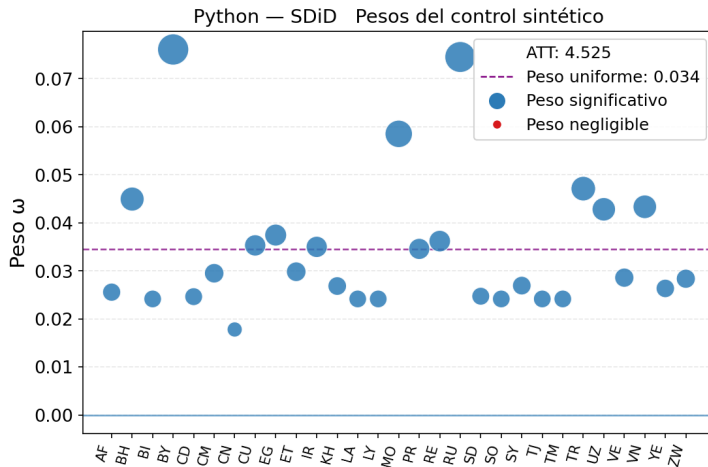
La significancia *bootstrap* no es la única evidencia que conviene revisar. Las pruebas de placebo refinan la lectura $\Rightarrow$ siguiente sección.

Tendencias y contrafactual: Python



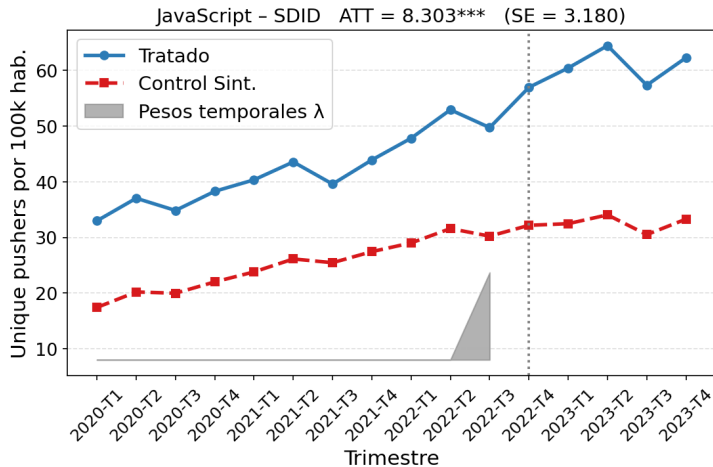
Tratado (azul) vs. control sintético SDiD (rojo). Las trayectorias coinciden hasta Q4-2022; luego divergen. **ATT** = 4,525 unique pushers por 100k.

Pesos del control sintético: Python



Puntos azules: donantes con peso significativo ($\geq 30\%$ del peso uniforme). **Puntos rojos:** pesos negligibles. Línea morada: peso uniforme $1/N_c \approx 0,034$.

Tendencias y contrafactual: JavaScript



Atención: ligera pre-tendencia ascendente antes de Q4-2022 $\Rightarrow$ el efecto causal en JavaScript debe interpretarse con cautela pese a la alta significancia.

9. Robustez

Cuadro 5: se excluyen los 6 censores severos (CN, CU, IR, BY, RU, SY).

Lenguaje	SDiD completo	SDiD restringido
JavaScript	8,303***	8,143***
Python	4,525***	4,378***
TypeScript	3,078***	3,025***
Java	1,725**	1,734**
C	1,276***	1,235***
C++	1,131***	1,094***
PHP	0,916**	0,893**
Ruby	0,808***	0,781***
Go	0,720***	0,701***
C#	0,443	0,398

Los resultados son cualitativa y cuantitativamente similares.

Cuadro 6: dos pruebas no paramétricas por lenguaje.

Lenguaje	ATT obs.	Media plac.	p-valor plac.	LOO mín	LOO máx
JavaScript	8,303	−0,161	0,069	4,642	9,322
Python	4,525	0,355	0,069	3,553	5,111
TypeScript	3,078	0,022	0,069	1,904	3,598
Java	1,725	−0,065	0,103	1,072	2,646
C	1,276	0,039	0,103	1,095	1,436
C++	1,131	0,071	0,138	1,016	1,278
PHP	0,916	−0,243	0,172	0,415	1,124
Ruby	0,808	−0,009	0,034	0,667	0,853
Go	0,720	−0,007	0,034	0,587	0,839
C#	0,443	−0,010	0,207	0,411	0,824

Placebos in-space: 29 placebos por lenguaje. LOO: 29 reestimaciones eliminando un control a la vez.

Tres niveles de evidencia

Fuerte ($p \leq 0,05$)

- Go ($p = 0,034$)
- Ruby ($p = 0,034$)

ATT atípico vs. placebos.

Moderada ($p \leq 0,10$)

- JavaScript (0,069)
- Python (0,069)
- TypeScript (0,069)

Lenguajes priorizados por la hipótesis.

Débil ($p > 0,10$)

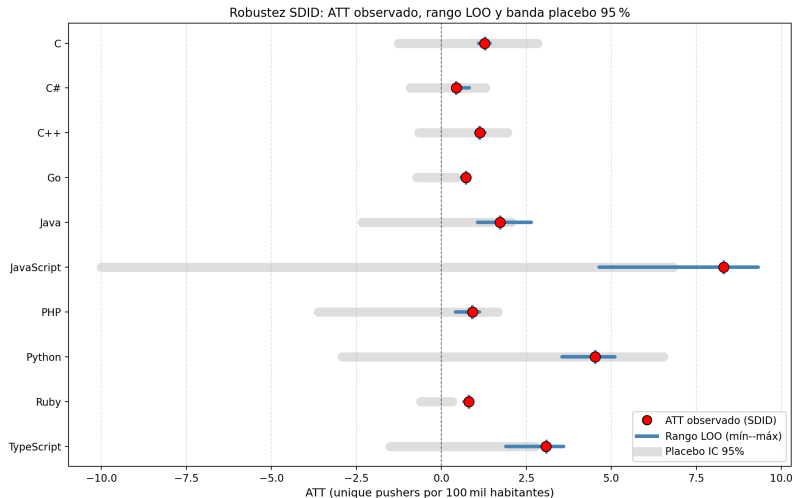
- C, C++, Java
- PHP
- C#

Cautela en interpretación causal.

Lectura honesta

La significancia *bootstrap* para C, C++ y Java coexiste con p-valores placebo $> 0,10$. Esto sugiere heterogeneidad estructural no completamente absorbida por el contrafactual.

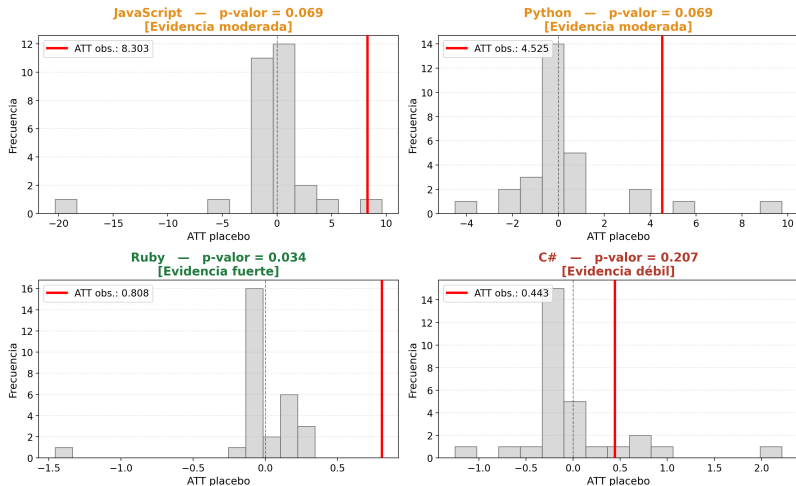
Forest plot: visualización conjunta



Punto rojo: ATT observado. **Barra azul:** rango LOO. **Banda gris:** intervalo placebo 95 %.

Distribución placebo: 4 lenguajes representativos

Distribución de ATT placebo — 4 lenguajes representativos



Línea roja: ATT observado. Histograma gris: 29 ATT placebo construidos en el grupo de control.

10. Limitaciones y conclusiones

- 1 **Asignación no aleatoria al control.** Los países sin ChatGPT son geopolíticamente distintos. El Cuadro 5 mitiga al excluir censores severos, pero no elimina el sesgo.
- 2 **Interferencia por VPN.** Una fracción de desarrolladores en países restringidos pudo haber accedido vía VPN. Esto **atenúa** el efecto estimado hacia cero.
- 3 **Ventana postratamiento corta (4 trimestres).** Limita inferencias de largo plazo y exige cautela frente a tendencias generales contemporáneas (Bard, Copilot Chat, Claude).
- 4 **Outcome limitado.** *Unique pushers* mide cantidad, no calidad ni intensidad. Países como China usan Gitee $\Rightarrow$ posible subestimación adicional en el control.
- 5 **Pre-tendencias en JavaScript.** El análisis de evento revela ligera pre-tendencia $\Rightarrow$ interpretación causal con cautela.

Hallazgo

La disponibilidad de ChatGPT incrementó la actividad de desarrollo de software en los países con acceso, con **efecto positivo y robusto** en los lenguajes priorizados por la hipótesis teórica: JavaScript, Python y TypeScript.

Apoyo teórico:

- Consistente con **Acemoglu & Restrepo (2018)**: reasignación de tareas hacia segmentos con menor productividad pre-tecnológica.
- Consistente con **Stigler (1961)**: *shock* negativo sobre costos de búsqueda.

Validación empírica:

- Tres métodos econométricos (DiD, CS, SDiD) coinciden en el signo para lenguajes principales.
- Pruebas de placebo y LOO sostienen la interpretación causal en el subconjunto robusto.

Conclusiones (2): heterogeneidad

Lenguajes	Evidencia	Interpretación
Go, Ruby	Fuerte	Efecto causal claramente identificado
JavaScript, Python, TypeScript	Moderada	Lenguajes priorizados por H2; efecto plausible
C, C++, Java	Débil	Significancia <i>bootstrap</i> pero no en placebos
PHP, C#	Débil	Efectos pequeños o ausentes

Las recomendaciones de política se fundamentan en el subconjunto con evidencia fuerte o moderada.

Recomendación 1: Acceso subsidiado para desarrolladores junior

Incentivos a empresas tecnológicas para financiar suscripciones **premium** (ChatGPT Plus) por períodos iniciales de **6 meses renovables** a desarrolladores junior orientados a ciencia de datos.

Recomendación 2: Bonos por productividad asistida por IA

Sistema de **bonos económicos** a trabajadores que evidencien mejoras de rendimiento atribuibles al uso de herramientas de IA, con seguimiento bimensual y estándares de QA.

Ambas medidas son consistentes con el mecanismo identificado por Acemoglu & Restrepo (2018): la IA expande la frontera productiva especialmente entre los segmentos del mercado laboral con menor productividad pre-tecnológica.

Contribución de esta tesis:

- Primer estudio que estima el efecto causal de ChatGPT sobre la **actividad de desarrollo de software** desagregada por lenguaje de programación.
- Aplicación del estimador SDiD (Arkhangelsky et al., 2021) en un contexto con $N_{tr} = 130$ unidades tratadas.
- Set completo de pruebas de robustez: placebos in-space + LOO donor pool + grupo de control restringido.

Líneas de investigación futuras:

- 1 Ventana postratamiento extendida hasta 2025-2026.
- 2 Análisis formal de mecanismos: correlación entre efecto y representación del lenguaje en datos de entrenamiento.
- 3 Heterogeneidad por nivel de ingreso, infraestructura digital y composición educativa.
- 4 Outcomes complementarios: commits, líneas de código, repositorios creados.

Gracias.

Quedamos atentos a sus consultas y comentarios.

Jesus Alberto Nicho Rosado • Karl Willem Janampa Aparicio

Asesor: Alexander Wilder Quispe Rojas

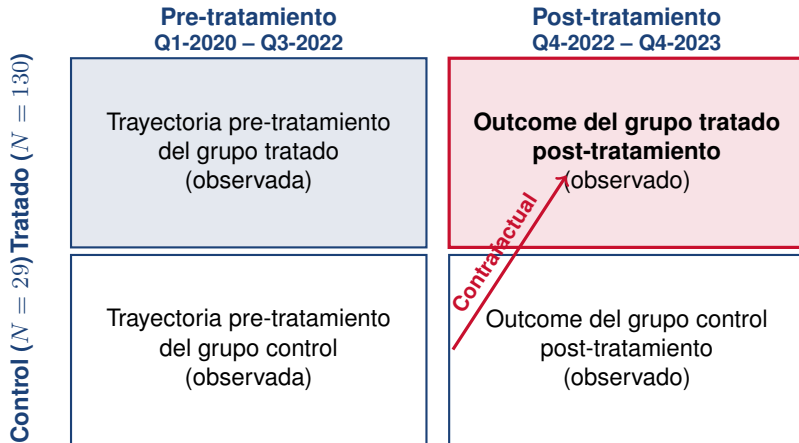
Pontificia Universidad Católica del Perú • Lima, 2026

Apéndice A

Slides de respaldo para preguntas

- A.1 Diseño cuasi-experimental (matriz 2×2)
- A.2 Cuadro 4 — Resultados con variables de control
- A.3 Event study SDID — Python (pre-tendencia plana)
- A.4 Event study SDID — TypeScript (pre-tendencia plana)
- A.5 Event study SDID — JavaScript (contraste, pre-tendencia visible)
- A.6 Pesos del control sintético — JavaScript

A.1 Diseño cuasi-experimental (matriz 2×2)



$$ATT = (\text{Tratado}_{\text{post}} - \text{Tratado}_{\text{pre}}) - (\text{Control}_{\text{post}} - \text{Control}_{\text{pre}}).$$

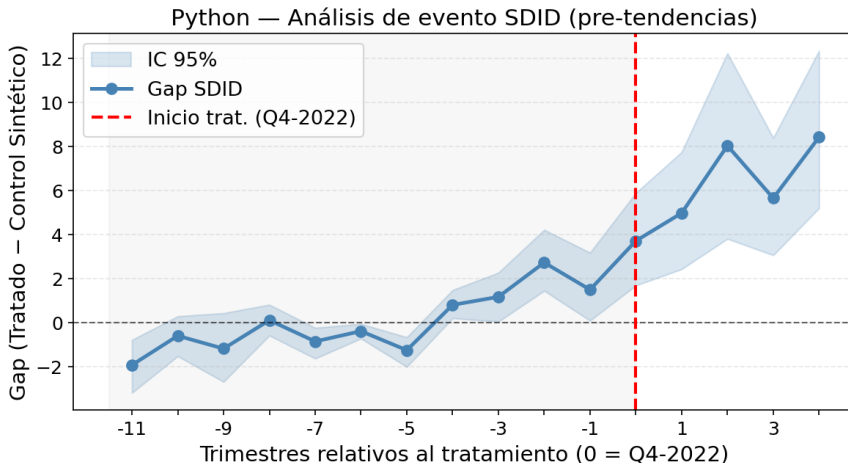
El SDID pondera las celdas para construir un contrafactual más preciso que el DiD clásico.

A.2 Resultados con variables de control (Cuadro 4)

Lenguaje	DiD	CS	SDiD
JavaScript	14,106*** (2,856)	10,470** (4,738)	9,142*** (3,228)
Python	7,831*** (1,147)	2,028 (3,531)	4,700*** (1,324)
TypeScript	7,189*** (0,846)	2,742 (2,244)	3,058*** (0,870)
Java	2,831*** (0,832)	2,224 (2,038)	1,726** (0,874)
C	2,774*** (0,367)	1,433 (1,240)	1,396*** (0,424)
C++	2,236*** (0,419)	1,242 (0,979)	1,157*** (0,437)
C#	1,475*** (0,383)	1,049 (1,252)	0,422 (0,386)
Go	1,411*** (0,176)	0,689 (0,606)	0,728*** (0,270)
PHP	0,994* (0,523)	0,974 (1,029)	1,034** (0,470)
Ruby	0,312 (0,272)	1,605* (0,855)	0,864*** (0,278)

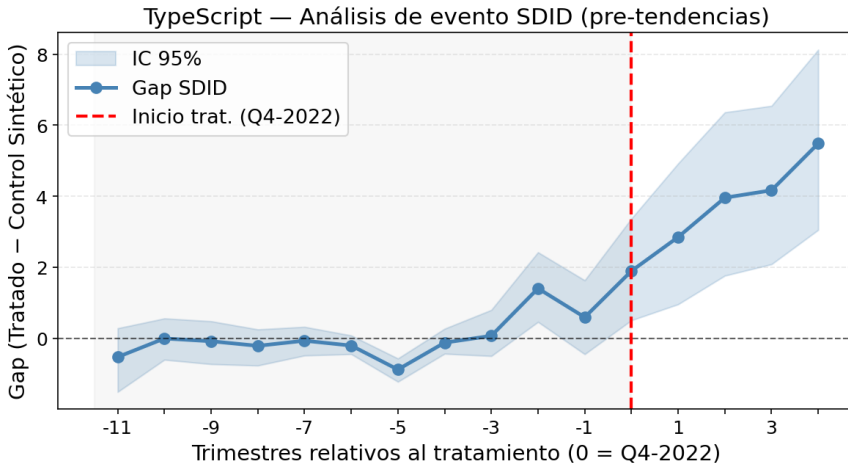
Controles: % individuos que usan computadora y % que usan internet (Banco Mundial). **Los efectos se mantienen e incluso se fortalecen.**

A.3 Event study SDID — Python



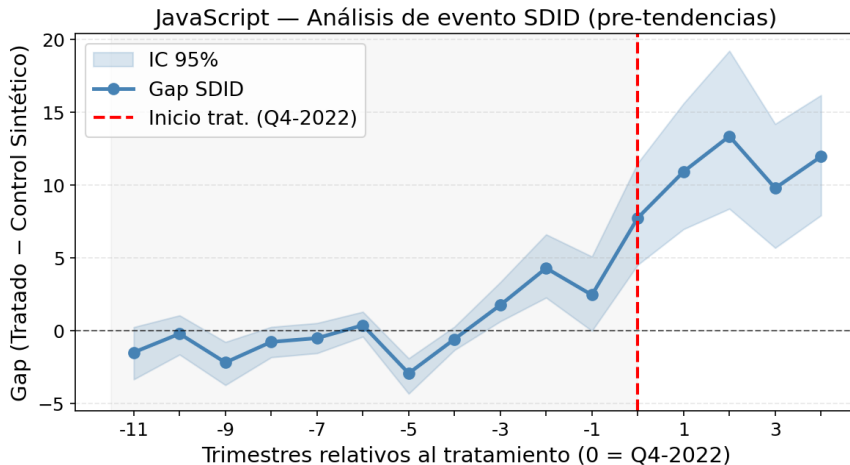
Pre-tendencia plana (gap cerca de cero antes de Q4-2022): el supuesto de paralelismo se cumple para Python $\Rightarrow$ el SDID estima el efecto causal de manera consistente.

A.4 Event study SDID — TypeScript



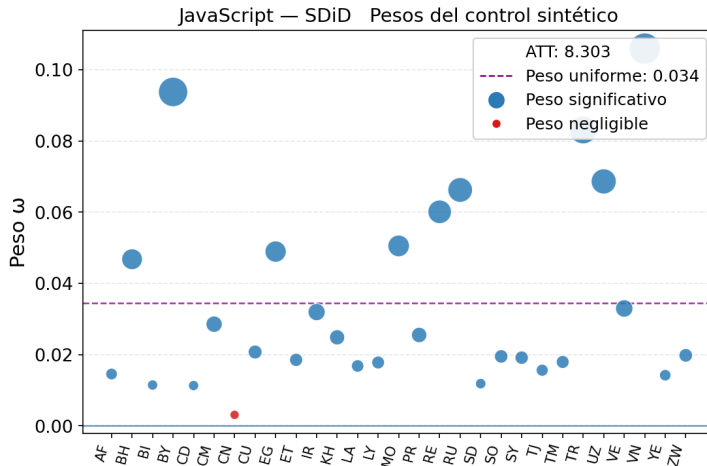
Pre-tendencia plana también para TypeScript. El efecto post-tratamiento es claro y la divergencia ocurre a partir de Q4-2022.

A.5 Event study SDID — JavaScript (contraste)



Atención: en JavaScript se observa una ligera pre-tendencia creciente. El problema es **específico de este lenguaje**, no del método (ver A.3 y A.4 con pre-tendencias planas).

A.6 Pesos del control sintético — JavaScript



Puntos azules: donantes con peso significativo ($\geq 30\%$ del peso uniforme). El contrafactual de JavaScript se construye con un conjunto diversificado de donantes, lo que reduce dependencia de cualquier país individual.

Apéndice B

Derivación formal del marco teórico — 8 pasos

- B.1 Función de producción a nivel de tarea (Paso 1)
- B.2 Conjunto de tareas automatizables (Paso 2)
- B.3 Ahorro relativo por tarea (Paso 3)
- B.4 Ganancia individual: partición con/sin IA (Paso 4.1–4.2)
- B.5 Ganancia exacta y reescritura (Paso 4.3–4.4)
- B.6 Identidad de covarianza (Paso 4.5)
- B.7 Normalización de escala (Paso 4.6)
- B.8 Decisión de entrada al mercado (Paso 5)
- B.9 Agregación a desarrolladores activos (Paso 6)
- B.10 Per cápita, absorción y separabilidad (Paso 7)
- B.11 Especificación de panel con efectos fijos (Paso 8)
- B.12 Resumen paso a paso

En **naranja** se marcan los pasos que dependen de un supuesto no trivial.

B.1 Función de producción a nivel de tarea (Paso 1)

El desarrollador elige el método más productivo en cada tarea $z \in [0, 1]$, y la producción total integra sobre el continuo de tareas:

$$y_{jc}(z; \mu_j) = \max\{h_c(z), \mu_j a_c(z)\}, \quad y_{jc}(\mu_j) = \int_0^1 \max\{h_c(z), \mu_j a_c(z)\} dz.$$

- $\mu_j = 0$ (sin IA): $y_{jc}(0) = \int_0^1 h_c(z) dz$ (pues $h_c(z) > 0$).
- $\mu_j = 1$ (con IA): $y_{jc}(1) = \int_0^1 \max\{h_c, a_c\} dz \geq y_{jc}(0)$ puntualmente.

Regularidad (mantenida en toda la derivación): h_c, a_c medibles e integrables en $[0, 1]$ y $\mathcal{Z}_c^{\text{auto}}$ medible —para que las integrales existan—; F_i continuamente diferenciable cerca del umbral (necesario en el Paso 6).

B.2 Conjunto de tareas automatizables (Paso 2)

Tareas con ganancia marginal estricta de la IA, y reescritura del máximo con la indicadora:

$$\mathcal{Z}_c^{\text{auto}} = \{z : a_c(z) > h_c(z)\}, \quad \text{máx}\{h_c, a_c\} = h_c \mathbf{1}[z \notin \mathcal{Z}_c^{\text{auto}}] + a_c \mathbf{1}[z \in \mathcal{Z}_c^{\text{auto}}].$$

Comprobación por casos:

- $z \in \mathcal{Z}_c^{\text{auto}}$: $a_c > h_c$, el máx. es a_c ; indicadoras 1 y 0. ✓
- $z \notin \mathcal{Z}_c^{\text{auto}}$: $a_c \leq h_c$, el máx. es h_c ; indicadoras 0 y 1. ✓

Medida de tareas automatizables: $\tau_c = \int_0^1 \mathbf{1}[a_c > h_c] dz \in [0, 1]$.

B.3 Ahorro relativo por tarea automatizable (Paso 3)

Para $z \in \mathcal{Z}_c^{\text{auto}}$, el ahorro relativo que aporta la IA:

$$s_c(z) = \frac{a_c(z) - h_c(z)}{h_c(z)} > 0 \quad (\text{pues } a_c > h_c > 0).$$

Ahorro promedio *condicional* a las tareas automatizables:

$$\rho_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} s_c(z) dz, \quad \tau_c > 0.$$

Nota: si $\tau_c = 0$ no hay tareas automatizables y el efecto se interpreta como nulo.

B.4 Ganancia individual: partición con/sin IA (Paso 4.1–4.2)

Sustituyendo la reescritura con indicadoras y usando $\int_0^1 f \mathbf{1}[z \in A] dz = \int_A f dz$:

$$\text{Con IA: } y_{jc}(1) = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c dz + \int_{\mathcal{Z}_c^{\text{auto}}} a_c dz,$$

$$\text{Sin IA: } y_{jc}(0) = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c dz + \int_{\mathcal{Z}_c^{\text{auto}}} h_c dz.$$

Ambas comparten exactamente el término sobre las tareas *no* automatizables.

B.5 Ganancia exacta y reescritura (Paso 4.3–4.4)

Al restar, el término de tareas no automatizables se **cancela**:

$$\Delta y_{jc} = y_{jc}(1) - y_{jc}(0) = \int_{\mathcal{Z}_c^{\text{auto}}} [a_c(z) - h_c(z)] dz.$$

Usando $s_c = \frac{a_c - h_c}{h_c} \Rightarrow a_c - h_c = h_c s_c$:

$$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) s_c(z) dz$$

Nota: hasta aquí todo es **identidad exacta** — la derivación no ha impuesto ningún supuesto.

B.6 Identidad de covarianza (Paso 4.5)

Definiendo los promedios condicionales sobre $\mathcal{Z}_c^{\text{auto}}$, $\bar{h}_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} h_c dz$ y $\rho_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} s_c dz$, y aplicando la **identidad de covarianza**:

$$\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] = \mathbb{E}[h_c] \mathbb{E}[s_c] + \text{Cov}(h_c, s_c) \Rightarrow \Delta y_{jc} = \tau_c [\bar{h}_c \rho_c + \text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c)].$$

Supuesto 1 — covarianza cero

$$\text{Imponer } \text{Cov}(h_c, s_c) = 0 \Rightarrow \Delta y_{jc} = \tau_c \rho_c \bar{h}_c.$$

Nota: es el **primer** supuesto no trivial; exige que productividad humana h_c y ahorro relativo s_c no estén correlacionados dentro de $\mathcal{Z}_c^{\text{auto}}$.

B.7 Normalización de escala (Paso 4.6)

Se elige la unidad de producción tal que el promedio de productividad humana en las tareas automatizables sea unitario, $\bar{h}_c = 1$:

$$\Delta y_{jc} = \tau_c \rho_c$$

Es decir, la ganancia individual es el producto **fracción automatizable** (τ_c) $\times$ **ahorro relativo promedio** (ρ_c), que más adelante define la exposición del lenguaje θ_c . *Nota:* no es un resultado empírico, es una **elección de escala**.

B.8 Decisión de entrada al mercado (Paso 5)

Utilidad de programar $U_{\text{prog}} = \eta + y_{jc}(\mu_j)$ frente a la reserva $U_{\text{out}} = r_i$; se entra si $\eta + y_{jc}(\mu_j) \geq r_i$. Umbral mínimo de habilidad:

$$\eta_c^*(\mu) = r_i - y_{jc}(\mu) \implies \eta_c^*(1) = \eta_c^*(0) - \underbrace{\tau_c \rho_c}_{\Delta y_{jc}}.$$

El acceso a IA **reduce** el umbral de habilidad $\Rightarrow$ entran programadores marginales.

Nota: con la covarianza exacta, $\eta_c^*(1) = \eta_c^*(0) - \tau_c [\bar{h}_c \rho_c + \text{Cov}(h_c, s_c)]$.

B.9 Agregación a desarrolladores activos (Paso 6)

Con población N_i y CDF de habilidad F_i (densidad f_i):

$$\text{Devs}_{ic}(\mu) = N_i [1 - F_i(\eta_c^*(\mu))].$$

Taylor de primer orden alrededor de $\eta_c^*(0)$, con $\Delta\eta = \tau_c \rho_c$:

$$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) \approx N_i f_i(\eta_c^*(0)) \tau_c \rho_c.$$

Nota: la aproximación de Taylor es válida si $\Delta\eta = \tau_c \rho_c$ es pequeño frente a la escala en que cambia la densidad f_i .

B.10 Per cápita, absorción y separabilidad (Paso 7)

Resultado por cada 100 000 habitantes:

$$\Delta Y_{ic} = \frac{\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0)}{N_i} \cdot 10^5 \approx \underbrace{10^5 f_i(\eta_c^*(0))}_{\phi_i} \cdot \underbrace{\tau_c \rho_c}_{\theta_c} \equiv \phi_i \theta_c.$$

Supuesto 4 — separabilidad país–lenguaje

Como $\eta_c^*(0) = r_i - y_{jc}(0)$ depende de c , en rigor $\phi_{ic} = 10^5 f_i(\eta_c^*(0))$. Escribir ϕ_i (solo país) exige $f_i(\eta_c^*(0)) \approx f_i$ (densidad plana en c), o equivalentemente que las productividades base $y_{jc}(0)$ sean similares entre lenguajes.

Nota: sin separabilidad el ATT por lenguaje sigue identificado, pero como $\theta_c \bar{\phi}_{\mathcal{T},c}$ (absorción promedio del lenguaje).

B.11 Especificación de panel con efectos fijos (Paso 8)

Descomponiendo la línea base $Y_{ic}(0) = \alpha_i + \beta_t + \delta_c + \eta_{ict}$ y sustituyendo $\Delta Y_{ic} \approx \theta_c \phi_i A_{it}$ (con $A_{it} \in \{0, 1\}$):

$$Y_{ict} = \alpha_i + \beta_t + \delta_c + \theta_c \phi_i A_{it} + \eta_{ict}$$

Para interpretación *causal* se requiere exogeneidad condicional:

$$\mathbb{E}[\eta_{ict} \mid \alpha_i, \beta_t, \delta_c, A_{it}] = 0.$$

Lectura económica: el efecto de tratamiento $\theta_c \phi_i$ es **multiplicativo** —máximo donde coinciden un lenguaje muy expuesto (θ_c alto) y un país con gran masa cerca del umbral (ϕ_i alto); si una de las dos es baja, el efecto se atenúa.

B.12 Resumen paso a paso

Paso	Resultado	Tipo
1	$y_{jc}(\mu_j) = \int_0^1 \max\{h_c, \mu_j a_c\} dz$	Identidad / definición
2	$\tau_c = \int_0^1 \mathbf{1}[a_c > h_c] dz$	Definición exacta
3	$\rho_c = \frac{1}{\tau_c} \int s_c dz$	Definición exacta ($\tau_c > 0$)
4.1–4.4	$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} h_c s_c dz$	Identidad exacta
4.5	$\Delta y_{jc} = \tau_c \rho_c \bar{h}_c$	Supuesto: $\text{Cov}(h_c, s_c) = 0$
4.6	$\Delta y_{jc} = \tau_c \rho_c$	Normalización $\bar{h}_c = 1$
5	$\eta_c^*(1) = \eta_c^*(0) - \tau_c \rho_c$	Depende del Paso 4
6	$\approx N_i f_i(\eta_c^*(0)) \tau_c \rho_c$	Taylor 1.º orden
7	$\Delta Y_{ic} \approx \theta_c \phi_i$	Aprox. + separabilidad $\phi_{ic} \approx \phi_i$
8	$Y_{ict} = \alpha_i + \beta_t + \delta_c + \theta_c \phi_i A_{it} + \eta_{ict}$	Especificación de panel

En **naranja**, los pasos que dependen de un **supuesto no trivial**; el resto son identidades o definiciones exactas.